



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 12-b: Mi solución de Automata

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

11 de Septiembre, 2025

Ejercicio

Actividad

El código hace lo siguiente:

```
2 public class Automaton {
3     public bool IsWordValid(string fileDFA, string word) {
4         StreamReader reader = new StreamReader(fileDFA);
5
6         /* ... Lee los estados de aceptación ... */
7
8         /* ... Lee las transiciones ... */
9
10        int state = 0;
11        foreach (var symbol in word)
12            if(transitions.ContainsKey((state, symbol)))
13                state = transitions[(state, symbol)];
14
15        return acceptingStates.Contains(state);
16    }
17 }
```

Este código lee los estados de aceptación:

```
2 HashSet<int> acceptingStates = new HashSet<int>();
3 string firstLine = reader.ReadLine();
4 string[] acceptingStateStrs = firstLine.Split(',');
5 foreach (var acceptingStateStr in acceptingStateStrs) {
6     int acceptingState = Convert.ToInt32(acceptingStateStr);
7     acceptingStates.Add(Convert.ToInt32(acceptingState));
8 }
```

Este código lee las transiciones:

```
10 Dictionary<(int, char), int> transitions = new ();
11 while (!reader.EndOfStream){
12     string transitionStr = reader.ReadLine();
13     string[] transitionComponents = transitionStr.Split(',');
14     int startingState = Convert.ToInt32(transitionComponents[0]);
15     char symbol = Convert.ToChar(transitionComponents[1]);
16     int endingState = Convert.ToInt32(transitionComponents[2]);
17     transitions[(startingState, symbol)] = endingState;
18 }
```

Finalmente, el código cuenta con tests (que podría ser necesario modificar):

```
2 [Theory]
3 [InlineData("01.txt", "abc")]
4 [InlineData("01.txt", "sdwaagfbfscds")]
5 [InlineData("02.txt", "")]
6 [InlineData("02.txt", "aaa")]
7 public void IsWordValid_ThisWordShouldBeValid(string fileDFA, string
    word){
8     fileDFA = GetPathToDFA(fileDFA);
9     Automaton dfa = new Automaton();
10
11     bool isWordValid = dfa.IsWordValid(fileDFA, word);
12
13     Assert.True(isWordValid);
14 }
```

Solución Propuesta

Solución propuesta

Les propongo esta solución que cuenta con clases cohesivas:

Solución propuesta

Les propongo esta solución que cuenta con clases cohesivas:

```
2 [Theory]
3 [InlineData("01.txt", "abc")]
4 [InlineData("01.txt", "sdwaagfbfscds")]
5 [InlineData("02.txt", "")]
6 [InlineData("02.txt", "aaa")]
7 public void IsWordValid_ThisWordShouldBeValid(string fileDFA, string
    word){
8     fileDFA = GetPathToDFA(fileDFA);
9     Automaton dfa = new Automaton();
10
11     bool isWordValid = dfa.IsWordValid(fileDFA, word);
12
13     Assert.True(isWordValid);
14 }
```

Solución propuesta

Les propongo esta solución que cuenta con clases cohesivas:

```
2 [Theory]
3 [InlineData("01.txt", "abc")]
4 [InlineData("01.txt", "sdwaagfbfscds")]
5 [InlineData("02.txt", "")]
6 [InlineData("02.txt", "aaa")]
7 public void IsWordValid_ThisWordShouldBeValid(string fileDFA, string
    word){
8     fileDFA = GetPathToDFA(fileDFA);
9     Automaton dfa = _reader.readFromFile(fileDFA);
10
11     bool isWordValid = dfa.IsWordValid(word);
12
13     Assert.True(isWordValid);
14 }
```

```
2 public class AutomatonReader {
3     private string[] _lines;
4     private States _acceptingStates;
5     private Transitions _transitions;
6
7     public Automaton ReadFromFile(string file) {
8         _lines = File.ReadAllLines(file);
9         DecodeAcceptingStates();
10        DecodeTransitions();
11        return new Automaton(_transitions, _acceptingStates);
12    }
13    private void DecodeAcceptingStates() {
14        StateDecoder decoder = new StateDecoder();
15        string acceptingStatesInfo = _lines[0];
16        _acceptingStates = decoder.DecodeFrom(acceptingStatesInfo);
17    }
18    private void DecodeTransitions() {
19        TransitionDecoder decoder = new TransitionDecoder();
20        string[] transitionInfos = _lines[1..];
21        _transitions = decoder.DecodeFrom(transitionInfos);
22    }
23 }
```

Solución propuesta

```
2 public class States {  
3     private readonly HashSet<int> _states = new ();  
4  
5     public void Add(int state)  
6         => _states.Add(state);  
7  
8     public bool Contains(int state)  
9         => _states.Contains(state);  
10 }
```

Solución propuesta

```
2 public class Transitions {  
3     private readonly Dictionary<(int, char), int> _transitions = new ();  
4  
5     public void AddTransition(int initialState, char symbol, int  
6         finalState)  
7         => _transitions[(initialState, symbol)] = finalState;  
8  
9     public int GetNextState(int initialState, char symbol) {  
10         (int, char) key = (initialState, symbol);  
11         if (_transitions.ContainsKey(key))  
12             return _transitions[key];  
13         return initialState;  
14     }  
}
```

```
2 public class AutomatonReader {
3     private string[] _lines;
4     private States _acceptingStates;
5     private Transitions _transitions;
6
7     public Automaton ReadFromFile(string file) {
8         _lines = File.ReadAllLines(file);
9         DecodeAcceptingStates();
10        DecodeTransitions();
11        return new Automaton(_transitions, _acceptingStates);
12    }
13    private void DecodeAcceptingStates() {
14        StateDecoder decoder = new StateDecoder();
15        string acceptingStatesInfo = _lines[0];
16        _acceptingStates = decoder.DecodeFrom(acceptingStatesInfo);
17    }
18    private void DecodeTransitions() {
19        TransitionDecoder decoder = new TransitionDecoder();
20        string[] transitionInfos = _lines[1..];
21        _transitions = decoder.DecodeFrom(transitionInfos);
22    }
23 }
```

Solución propuesta

```
2 public class StateDecoder {
3     private States _states;
4
5     public States DecodeFrom(string encodedStates) {
6         _states = new States();
7         foreach (var stateInfo in encodedStates.Split(','))
8             AddState(stateInfo);
9         return _states;
10    }
11
12    private void AddState(string stateInfo) {
13        int state = Convert.ToInt32(stateInfo);
14        _states.Add(Convert.ToInt32(state));
15    }
16 }
```

Solución propuesta

```
2 public class TransitionDecoder {
3     private Transitions _transitions;
4
5     public Transitions DecodeFrom(string[] transitionInfos) {
6         _transitions = new Transitions();
7         foreach (string transitionInfo in transitionInfos)
8             AddTransition(transitionInfo);
9         return _transitions;
10    }
11
12    private void AddTransition(string transitionInfo) {
13        string[] transitionComponents = transitionInfo.Split(',');
14        int initialState = Convert.ToInt32(transitionComponents[0]);
15        char symbol = Convert.ToChar(transitionComponents[1]);
16        int finalState = Convert.ToInt32(transitionComponents[2]);
17        _transitions.AddTransition(initialState, symbol, finalState);
18    }
19 }
```



```
2 public class AutomatonReader {
3     private string[] _lines;
4     private States _acceptingStates;
5     private Transitions _transitions;
6
7     public Automaton ReadFromFile(string file) {
8         _lines = File.ReadAllLines(file);
9         DecodeAcceptingStates();
10        DecodeTransitions();
11        return new Automaton(_transitions, _acceptingStates);
12    }
13    private void DecodeAcceptingStates() {
14        StateDecoder decoder = new StateDecoder();
15        string acceptingStatesInfo = _lines[0];
16        _acceptingStates = decoder.DecodeFrom(acceptingStatesInfo);
17    }
18    private void DecodeTransitions() {
19        TransitionDecoder decoder = new TransitionDecoder();
20        string[] transitionInfos = _lines[1..];
21        _transitions = decoder.DecodeFrom(transitionInfos);
22    }
23 }
```

```
2 public class Automaton {
3     private States acceptingStates;
4     private Transitions transitions;
5     private const int InitialState = 0;
6
7     public Automaton(Transitions transitions, States acceptingStates) {
8         this.transitions = transitions;
9         this.acceptingStates = acceptingStates;
10    }
11
12    public bool IsWordValid(string word) {
13        int state = InitialState;
14        foreach (char symbol in word)
15            state = transitions.GetNextState(state, symbol);
16        return acceptingStates.Contains(state);
17    }
18 }
```

```
2 public class Automaton {
3     private States acceptingStates;
4     private Transitions transitions;
5     private const int InitialState = 0;
6
7     public Automaton(Transitions transitions, States acceptingStates) {
8         this.transitions = transitions;
9         this.acceptingStates = acceptingStates;
10    }
11
12    public bool IsWordValid(string word) {
13        int state = InitialState;
14        foreach (char symbol in word)
15            state = transitions.GetNextState(state, symbol);
16        return acceptingStates.Contains(state);
17    }
18 }
```

... y eso es todo, ahora el código funciona en base a clases con una responsabilidad :)

Solución propuesta

Nuestra clase Automaton (que hacía todo) la dividimos en las siguientes responsabilidades:

Nuestra clase Automaton (que hacía todo) la dividimos en las siguientes responsabilidades:

- Definir un conjunto de estados
- Definir un conjunto de transiciones
- Decodificar los estados de aceptación
- Decodificar las transiciones
- Crear el Autómata
- Definir el Autómata

Nuestra clase `Automaton` (que hacía todo) la dividimos en las siguientes responsabilidades:

- Definir un conjunto de estados
- Definir un conjunto de transiciones
- Decodificar los estados de aceptación
- Decodificar las transiciones
- Crear el Autómata
- Definir el Autómata

A partir de `Automaton` distinguimos 6 responsabilidades distintas